PROGRAMAÇÃO EM PYTHON

SENAI

PROGRAMAÇÃO EM PYTHON

Plano de Aula

Conteúdo:

- Try e Except;
- Manipulação de Arquivos;
- Atividades;

Inicio:

As informações deste conteúdo visam compreender o conteúdo do curso.

SENAI

PROGRAMAÇÃO EM PYTHON

Objetivo

Capacitar profissionais para desenvolver aplicações em linguagem Python, por meio de técnicas de programação, seguindo boas práticas, procedimentos e normas.

SENAI

PROGRAMAÇÃO EM PYTHON

Try e Except

O tratamento de exceções permite lidar com erros de forma controlada, evitando que o programa seja interrompido abruptamente. A estrutura **try** e **except** possibilita capturar e tratar esses erros, permitindo a criação de programas mais robustos e confiáveis.

```
resultado = 10 / 0
```

Erro no terminal:

```
Traceback (most recent call last):
  File "D:\Fabrica de Programadores\09-Try.py", line 1, in <module>
    resultado = 10 / 0
                ~~~~^~~
ZeroDivisionError: division by zero
```

Usando o try e except:

```
try:
    # Código que pode causar um erro
    resultado = 10 / 0
except:
    # Código que trata o erro
    print("Ocorreu um erro durante a operação!")
```

SENAI

PROGRAMAÇÃO EM PYTHON

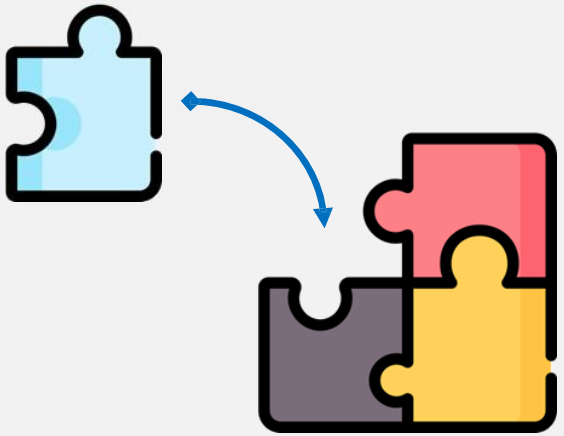
Try e Except

Conceitos Básicos

O que são exceções?

Exceções: São erros que ocorrem durante a execução do programa, como divisão por zero, acesso a índice inexistente, conversões inválidas, etc.

Objetivo: Permitir que o programa trate esses erros de maneira apropriada, exibindo mensagens amigáveis ou tomando ações corretivas.



SENAI

PROGRAMAÇÃO EM PYTHON

Try e Except

Por que utilizar try e except?

Prevenção de falhas: Evita que erros não tratados interrompam a execução do programa.

Melhoria da experiência do usuário: Permite fornecer feedback claro sobre o que deu errado.

Manutenção do fluxo de execução: Mesmo ocorrendo um erro, o programa pode continuar a execução ou realizar operações alternativas.

Estrutura Básica: try e except

Bloco try: Contém o código que pode gerar uma exceção.

Bloco except: É executado se ocorrer uma exceção no bloco try.

```
try:
    # Código que pode causar um erro
    resultado = 10 / 0
except:
    # Código que trata o erro
    print("Ocorreu um erro durante a operação!")
```

SENAI

PROGRAMAÇÃO EM PYTHON

Try e Except

Você pode estruturar o tratamento de exceções de diversas formas, permitindo capturar erros de maneira específica ou genérica. Eis um resumo das possibilidades e sintaxes:

```
try:
    # Bloco de código que pode gerar exceções
except ExceptionType:
    # Bloco para tratar a exceção específica "ExceptionType"
```

```
try:
    # código
except ValueError as ve:
    # Aqui, "ve" contém a exceção lançada.
```

Exceção Específica: Você pode indicar uma exceção específica, como ValueError, TypeError, etc.

SENAI

PROGRAMAÇÃO EM PYTHON

Try e Except

```python
num1 = input("Digite o primeiro número: ")
num2 = input("Digite o segundo número: ")

try:
    num1 = int(num1)
    num2 = int(num2)

    print(f"A soma dos números é: {num1 + num2}")
except:
    print("Digite um número correto!")
```

SENAI

PROGRAMAÇÃO EM PYTHON

Try e Except

Uso de Múltiplos Blocos

Você pode ter vários blocos except para tratar tipos diferentesde exceções:

```
try:
    # código que pode gerar exceções
except ValueError:
    # tratamento específico para ValueError
except TypeError:
    # tratamento específico para TypeError
except Exception as e:
    # captura qualquer outra exceção derivada de Exception
```

Ou uso de else e finally

```
try:
    # código sem exceção
except Exception:
    # tratamento de erro
else:
    # executado se não ocorrer nenhuma exceção
finally:
    # sempre executad, mesmo se houver uma exceção
```

SENAI

PROGRAMAÇÃO EM PYTHON

Try e Except

Tabela com alguns Except

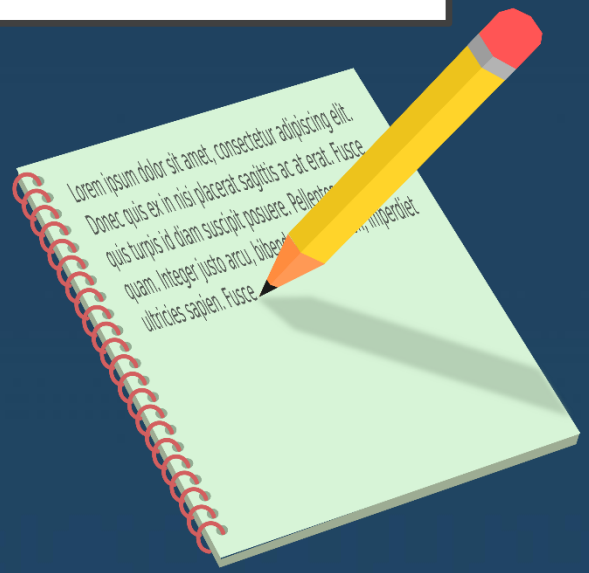
Exceção	Descrição
TypeError	Lançada quando uma operação ou função é aplicada a um objeto de tipo inadequado.
ValueError	Ocorre quando um valor possui tipo correto, mas é inapropriado para a operação.
KeyError	Gerada quando uma chave não existe em um dicionário.
BaseException	Classe base de todas as exceções.
Exception	Base para a maioria das exceções comuns; é o tipo que normalmente você captura.
ArithmeticError	Classe base para erros aritméticos, como divisão por zero ou overflow.
ZeroDivisionError	Ocorre quando há tentativa de divisão por zero.
FileNotFoundError	Erro gerado ao tentar acessar um arquivo ou diretório inexistente.
RuntimeError	Erro genérico que não se encaixa em outras categorias.
NotImplementedError	Usada para sinalizar que uma funcionalidade ou método ainda não foi implementado.
SyntaxError	Indica erros de sintaxe no código.
IndentationError	Subtipo de SyntaxError; ocorre quando há problemas na indentação.

SENAI

EXERCÍCIOS

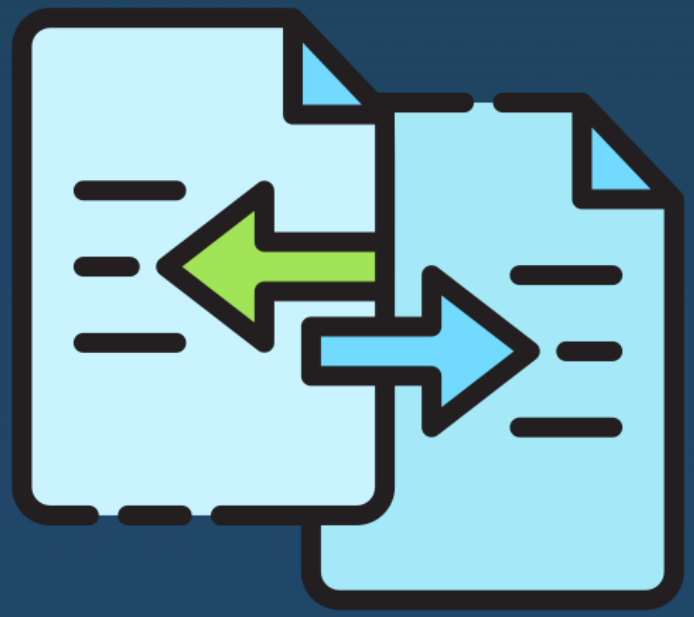
Aplique Try e Except nas atividades realizadas em sala de aula:

- Calculadora
- Cálculo de IMC
- Conversor de temperatura
- Conversor de moeda
- Tabuada



PROGRAMAÇÃO EM PYTHON

Manipulação de Arquivos



Usando a Função Built-in open()

A função open() é a maneira básica de abrir arquivos em Python.

Ela retorna um objeto que representa o arquivo e permite realizar operações como leitura, escrita e anexação de dados.

```
arquivo = open("caminho_do_arquivo.txt", "modo")
```

"caminho_do_arquivo.txt": Indica o caminho e o nome do arquivo a ser aberto.

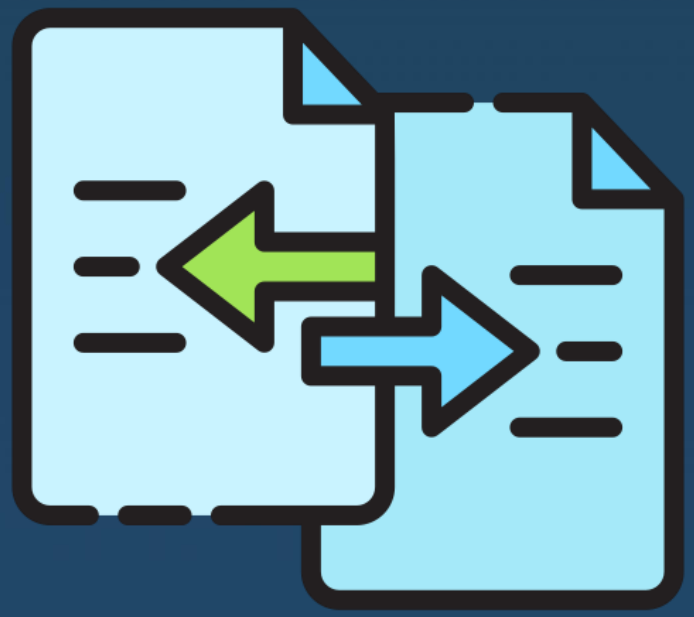
"modo": Especifica a forma como o arquivo será manipulado:

- `"r"` : Modo leitura (padrão). Gera erro se o arquivo não existir.
- `"w"` : Modo escrita. Cria um novo arquivo ou sobrescreve o existente.
- `"a"` : Modo anexação. Adiciona dados ao final do arquivo existente.
- `"r+"` : Modo leitura e escrita. O arquivo deve existir.

SENAI

PROGRAMAÇÃO EM PYTHON

Manipulação de Arquivos



Usando a Função Built-in open()

A sintaxe geral da função **open()** é:

```
arquivo = open("caminho_do_arquivo.txt", "modo")



# Podendo ser:

arquivo = open("caminho_do_arquivo.txt", "r")
# ou
arquivo = open("caminho_do_arquivo.txt", "w")
# ou
arquivo = open("caminho_do_arquivo.txt", "a")
```

SENAI

PROGRAMAÇÃO EM PYTHON

Manipulação de Arquivos

Contexto:

Vamos desenvolver um programa que solicitará ao usuário a inserção de dados para o cadastro de pessoas (**nome** e **e-mail**). Esses dados serão armazenados em um arquivo de texto, denominado "pessoa.txt".

A cada execução do programa, um novo registro será adicionado ao arquivo. Caso o arquivo não exista, ele será criado automaticamente.

O arquivo de texto será criado ao lado do script Python.

SENAI

PROGRAMAÇÃO EM PYTHON

Manipulação de Arquivos

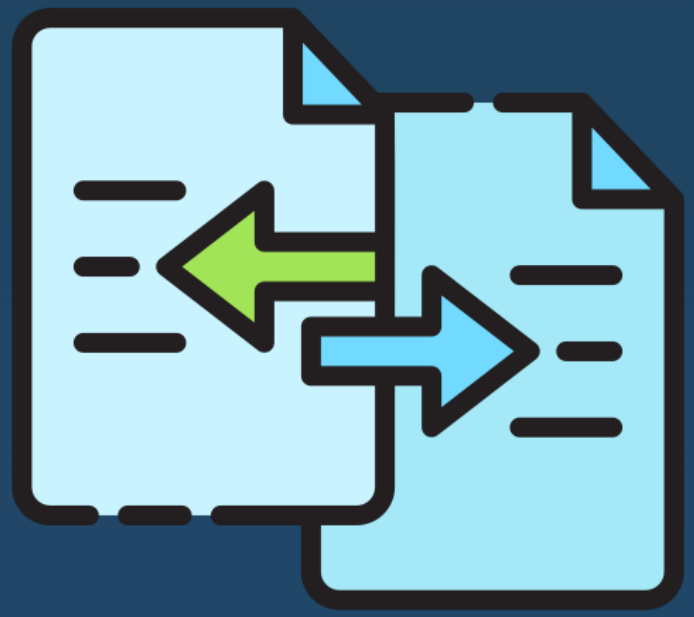
```
nome = input("Digite o nome: ")
email = input("Digite o e-mail: ")

arquivo = open("pessoa.txt", "a")
arquivo.write(nome + " | " + email + "\n")
arquivo.close()
```

- `arquivo = open("pessoa.txt", "a")` abre o arquivo "pessoa.txt" no modo de anexação ("a"), o que significa que os novos dados serão adicionados ao final do arquivo. Se o arquivo não existir, ele será criado automaticamente.
- `arquivo.write(nome + " | " + email + "\n")` escreve no arquivo uma linha contendo o nome e o e-mail separados pelo caractere " | ", finalizando com uma quebra de linha para que cada novo registro comece em uma linha separada.
- `arquivo.close()` fecha o arquivo, garantindo que todos os dados escritos sejam salvos e liberando os recursos utilizados durante a operação.

SENAI

PROGRAMAÇÃO EM PYTHON

Manipulação de Arquivos



Observação:

Esquecimentode fechar o arquivo utilizandoo método close() pode causar os seguintes problemas:

Perda de dados: Os dados podem ficar retidos no buffer e não serem gravados de forma definitiva no arquivo.

Uso desnecessário de recursos: O arquivo permanece aberto, consumindo recursos do sistema que poderiam ser liberados.

Acesso restrito: Outros programas ou mesmo novas operações dentro do seu programa podem ter dificuldade em acessar ou modificar o arquivo, pois ele pode permanecer bloqueado.

Possível corrupção do arquivo: Em casos extremos, se o programa for encerrado abruptamente, o arquivo pode ficar com informações incompletas ou corrompidas.

Para melhorar nosso script, vamos utilizar um comando reservado no Python que é **with**.

SENAI

PROGRAMAÇÃO EM PYTHON

Manipulação de Arquivos

```python
nome = input("Digite o nome: ")
email = input("Digite o e-mail: ")

with open("pessoa.txt", "a") as arquivo:
    arquivo.write(nome + " | " + email + "\n")
```

O comando with simplifica o gerenciamento de arquivos ao criar um "contexto" para as operações com o arquivo. Quando o bloco do with termina, o arquivo é fechado automaticamente, mesmo que ocorram erros durante a execução. Reduz a quantidade de código e a possibilidade de erros relacionados ao gerenciamento de recursos.

CURIOSIDADE

```python
import keyword
print(keyword.kwlist)
```

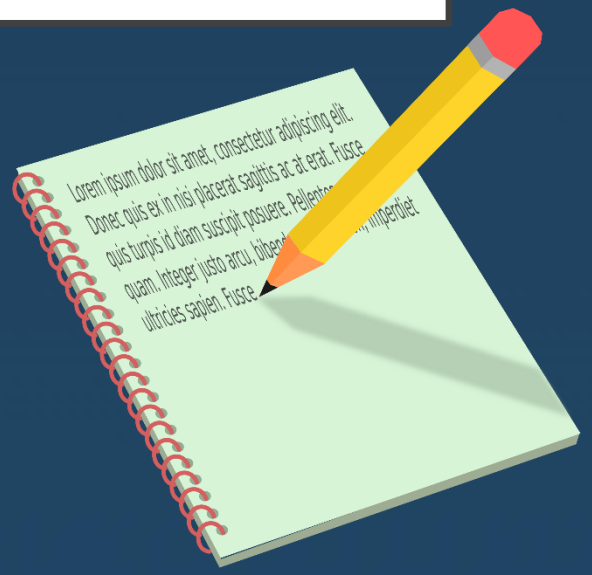
Para saber palavras reservadas da linguagem Python.

SENAI

EXERCÍCIOS

1 - Aplique manipulação de arquivos nas atividades realizadas em sala de aula, armazenando os dados trabalhados:

- Sistema de Avaliação de Desempenho Escolar
- Monitoramento de Saúde com Cálculo de IMC



2 – Desenvolva um programa que solicite ao usuário os dados de um produto (nome, valor e quantidade) e armazene-os em um arquivo de texto chamado "produtos.txt".

REFERÊNCIAS

ALVES, William Pereira. Lógica de programação de computadores. São Paulo: Érica, 2012.

ASCENCIO, Ana F. G.; CAMPOS, Edilene A. V. Fundamentos da programação de computadores. 3. ed. São Paulo: Pearson, 2012.

FARRELL, Joyce. Lógica e design de programação. São Paulo: Cengage Learning, 2010.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. Algoritmos: Lógica para desenvolvimento de programação de computadores. 27. ed. rev. São Paulo: Érica, 2014.

MEDINA, Marco; FERTIG, Cristina. Algoritmos e programação. Teoria e prática. São Paulo: Novatec, 2005.

PEREIRA, Silvio do Lago. Algoritmos e lógica de programação em C: Uma abordagem didática. São Paulo: Érica, 2010.

Importante:

Os conteúdos disponibilizados são específicos para este curso/turma, a divulgação ou reprodução do material para outras pessoas/organização não é autorizada.

ffigaro@sp.senai.br

Fernando Oliveira Figaro

Instrutor de Formação Profissional III

SENAI